

FlashMoE：在单一内核中的快速分布式混合专家

Osayamen Jonathan
Aimuyo*康奈尔大学
oja7@cornell.edu

Byungsoo Oh康奈尔大学
bo239@cornell.edu

Rachee Singh康奈尔大学
rs2293@cornell.edu

摘要

专家混合（MoE）模型的计算稀疏性使得随着模型规模增长，计算成本呈次线性增长，从而为训练海量神经网络提供了一条可扩展路径。然而，现有实现存在 GPU 利用率低、显著的延迟开销，以及无法利用任务局部性的根本性问题，其主要原因在于 CPU 管理的调度、主机发起的通信和频繁的内核启动。为克服这些局限性，我们开发了 FlashMoE，这是一种完全 GPU 驻留的 MoE 算子，将专家计算与 GPU 间通信融合到单个持久化 GPU 内核中。FlashMoE 支持对分发、计算与合并阶段进行细粒度流水化，从而消除启动开销并减少空闲间隙。不同于现有工作，FlashMoE 采用单边、设备发起的 GPU 间（R）DMA 传输，取代批同步集体通信，从而解锁载荷效率；在稀疏激活层中，我们消除臃肿或冗余的网络载荷。在 $8 \times \text{H100}$ GPU 节点上，对包含多达 128 名专家和 16K 标记序列的 MoE 模型进行评估时，FlashMoE 相比于当前最先进水平的基线方法，最高可实现 **9 $\times$** 更高的 GPU 利用率、**6 $\times$** 更低的延迟、**5.7 $\times$** 更高的吞吐量，以及 **4 $\times$** 更好的重叠效率——尽管我们使用 FP32，而基线使用 FP16。FlashMoE 表明，基于原则的 GPU 内核-硬件协同设计是突破大规模分布式机器学习性能上限的关键。代码见 <https://github.com/osayamenja/FlashMoE>。

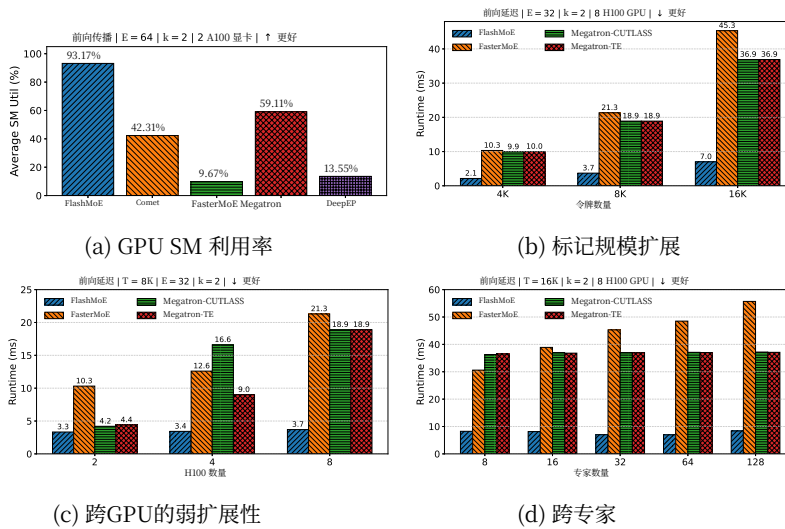


图1: FlashMoE 性能。

*通讯请联系 osayamen@stanford.edu

1 引言

当前最先进水平的大型语言模型 (LLMs) [1–5] 因其计算效率和在多种任务上的强劲性能而采用了专家混合 (MoE) 架构。传统的 Transformer 块由自注意力模块和后续的稠密前馈网络 (FFN) 组成 [6]。相比之下，专家混合架构将这一单个 FFN (图2(a)) 替换为多个尺寸相同的 FFN，称为专家 (图2(b))。一个可训练的神经网络，即门控函数，通过在运行时将输入标记动态路由到被选中的专家来稀疏地激活这些专家。由于引入了更多 FFN，模型参数增加，从而在不相应增加计算成本的情况下提升了模型质量。

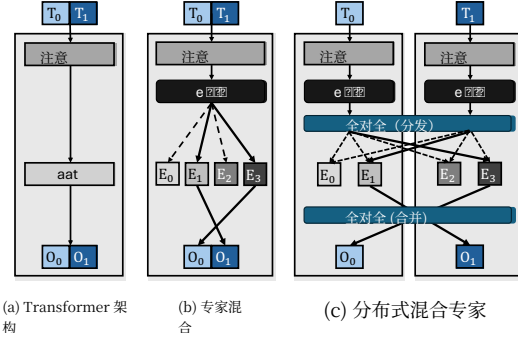


图2: Transformer 块 (a) 无专家混合, (b) 含专家混合, 以及 (c) 含分布式混合专家与专家并行。T、E 和 O 分别表示输入标记、专家和输出激活。

MoE 中的通信开销。As MoE

随着模型规模增长，GPU 内存限制使得无法在单个设备上承载所有专家。标准做法是使用专家并行 (EP) 将专家分布到多个 GPU 上，这需要通过诸如 *AlltoAll* [1, 4, 3, 7] (图2(c)) 等多对多通信原语进行令牌路由。另一轮 *AlltoAll* 通信将专家处理后的置换标记恢复到序列中的原始顺序。 *AlltoAll* 通信在 GPU 网络上难以优化，并且对掉队者延迟极为敏感——该现象指单个掉队的 GPU 会阻滞其他 GPU 的推进 [8]。这些通信操作可能占总运行时间的 68% [9, 10]，导致 GPU 处于空闲状态 (图3左上)。

MoE 中的内核启动开销。 为缓解这些通信瓶颈，近期工作将计算与通信内核流水化 (图3，左中)。然而，这些方案的有效性受到从 CPU 启动大量内核的开销限制。具体而言，现有实现 [11–14] *launch* 在单层的一次执行中会启动大量内核 (见表1)。频繁的内核启动通过以下方式对性能产生负面影响：(1) 在各个 GPU 上造成非确定的核函数启动时间，加剧慢节点问题；(2) 引入不必要的同步点，使 GPU 在继续前必须等待其他 GPU 或 CPU；(3) 在内核边界引发重复的全局内存往返。尽管 CUDA 图 [15] 可以在静态工作负载中部分缓解第一个问题，但它们与专家混合的动态专家路由模式不兼容。解决其余问题需要新的方案，我们在本文中通过完整的内核融合和异步设备发起的通信来实现。

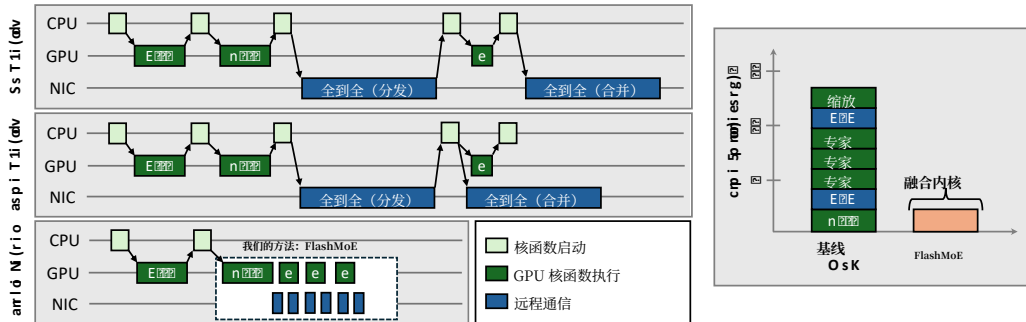


图3: 将 FlashMoE 与最先进技术进行比较，这些技术要么不让通信与计算重叠 (左，上)，要么有一定重叠 (左，中)。FlashMoE 是一个常驻内核，将 MoE 算子的全部计算与通信进行融合 (左，下)。FlashMoE 实现了设备发起的计算 (门控、专家前馈网络 (FFN)、缩放) 和通信任务 (右)。

我们的贡献：在单一内核中实现分布式混合专家。为克服当前最先进水平 MoE 模型中的这些根本性低效，我们开发了 FlashMoE，这是一种巨型内核，能够将

所有 MoE 计算和通信任务整合到单一持久化 GPU 内核中即，一个在整个 MoE 算子期间保持活跃的内核（图3 左下）。与由 CPU 协调的多个内核启动不同，FlashMoE 只需启动一次内核，显著减少 CPU 的参与。在该融合内核内，FlashMoE 实现了响应式编程模型，以在数以万计的 GPU 线程之间实现细粒度并行以及松耦合、非阻塞执行。

内核内块调度和瓦片并行性。 FlashMoE 实现了瓦片级并行性，即将输入标记矩阵划分为更小且彼此独立的单元，称为瓦片；这些单元由线程块进行处理，但由 warp（束）进行管理（调度与构建）。除一个外，我们将每个线程块专门化为处理器以执行计算。此外，我们指定一个专用的操作系统（OS）块（4 个 warp（束））来执行管理任务：（1）将计算工作调度到处理器（调度器）；（2）从其他 GPU 收到的消息中解码计算任务（订阅者）。这种设计使 FlashMoE 能够基于就绪度动态地将任务分配给 GPU 线程块，确保在专家混合的整个生命周期中没有 GPU SM（流式多处理器）处于空闲状态

算子。FlashMoE 选择 tile 尺寸，以最大化 GPU 的算术强度，同时受益于高度的并行性。

专家混合实现	GPU 算子
FlashMoE (本工作)	1
COMET [12]	33
Megatron-LM CUTLASS [13, 16]	85
Megatron-LM TE [13, 16]	261
Megatron-LM +DeepEP [1]	432
DeepSpeedMoE [11]	550

表1：我们使用 Nsight Systems [17] 对专家混合实现进行性能分析，报告其启动的 GPU 操作数量。我们统计在 2 块 A100 GPU 上的单个 MoE 层（门控 → 分发 → 专家 → 合并）中的操作数，其中每个 GPU 有 32 个专家。FlashMoE 首次将分布式混合专家的 MoE 层完全融合为单一 GPU 内核。

异步且负载高效的通信。 通过自底向上重新设计 MoE 算子，FlashMoE 解决了传统 MoE 执行流水线中固有的基础性低效问题。其中一个显著的低效是通信过程中的 token 填充。为简化编程复杂度，并由于集合通信 API 的对称性约束，现有实现必须对 token 载荷进行零填充，以匹配预定义的缓冲区大小。当 token 被非对称路由到专家时，就会出现这种情况，导致 GPU 接收到的远低于预期容量。然而，这些空载荷会浪费通信带宽，增加数据传输时延，并可能在某些实现中导致对空矩阵的不必要计算。FlashMoE 通过仅将未填充的 token 发送至拥有被选中的专家的 GPU，引入负载高效的通信，从而同时节省通信和计算资源。

技术挑战。 要实现 FlashMoE 的单内核设计并获得高性能，需要解决若干技术挑战：(1) 轻量级计算依赖管理；(2) 探索最优 SM 占用配置；(3) 实现设备内 BLAS 操作；(4) 将设备间与设备内同步开销最小化；(5) 在可用时，利用基于统一虚拟寻址（UVA）的 DMA 来实现传输感知。为应对这些挑战，FlashMoE 的设计与传统的同步 *AllToAll*（全互连）集体通信有着根本性的不同，在后者中，GPU 在层执行期间会出现显著的空闲时间。对于设备发起的通信，FlashMoE 使用 NVSHMEM [18] 以建立跨所有 GPU 的全局地址空间，用于直接内存访问（DMA）通信。对于设备内 BLAS，FlashMoE 通过 CUTLASS [19] 开发了定制的高性能 GEMM 运算。

结果。 我们的评估表明，FlashMoE 相较于最先进的实现，实现了 **6×** 时延加速比、**9×** 更高的 GPU 利用率、**4×** 更好的弱扩展效率，以及 **5.7×** 更高的吞吐量。我们预计这些性能提升在多节点场景中会进一步扩大，在该场景下，节点间通信通过低带宽的节点间链路进行（例如，RDMA、InfiniBand）。

2 动机

同步通信。 *AllToAll*（全互连）或 *AllGather*（全收集）通信在当前的 MoE 框架中是一个同步的集合操作，其完成需要所有相关 GPU 的参与。这里，GPU 之间的处理速度差异或内核调度差异会引发

算法 1: FlashMoE 分布式混合专家融合内核

输入: $A, O \in \mathbb{R}^{S \times H}$, $X \in \mathbb{R}^{E \times H \times D}$, N

```

1 开始
2    $T_\phi, G_\phi \leftarrow \text{融合门}(A)$ 
3   如果 块ID + 1 <  $N$  则
4     分发( $T_\phi, A$ )
5     processor::start()
6   else
7     如果 warpID == 0 则
8       scheduler::start()
9     else
10      订阅者::启动( $T_\phi, G_\phi, O, X$ )
11    结束 if
12  结束 if
13 结束

```

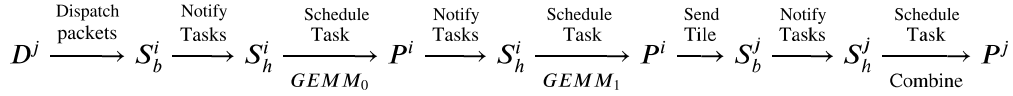


图 6: *DMoE Functional Dependencies Expressed as a Chain of Actor Interactions*。我们将 S_b 、 S_h 和 P 分别表示为订阅者、调度器和处理器参与者。对于任意 Actor $a \in \{S_b, S_h, P\}$, a^i 标识 GPU i 上的一个 Actor。我们将 D_i^j 定义为算子，其中 GPU j 向 GPU i 分发瓦片数据包。该图以瓦片粒度表达任务依赖，即 $GEMM_0$ 、 $GEMM_1$ 、合并 (combine) 和通信产生一个输出瓦片。通知以通过共享内存 (订阅者 $\leftrightarrow$ 调度器) 或全局内存 (调度器 $\leftrightarrow$ 处理器或 GPU 间通信) 传播的信号形式发生。注意，单边 GPU 间传输 (数据包或单个瓦片) 会 *coupled* 到一个信号上，以通知接收 GPU j 上的 S_b^j 消息已送达。

描述符 (§ 3.1)。在一个 GPU 上的 N 个线程块中，我们将 $N - 1$ 专用于担任 **Processor** 角色。我们将最后一个线程块专门用作操作系统 (OS)。在该线程块内，我们将三个 warp (束) 专用于 **Subscriber** 角色，将一个 warp (束) 专用于 **Scheduler** 角色。将线程块在各 Actor 之间如此划分是有意为之：我们的目标是为管理性任务使用尽可能少的资源，同时将大部分资源保留下来用于执行专家混合 (MoE) 计算任务。图 5 概述了 FlashMoE 架构及其组成的 Actor，而 算法 1 则以代码给出了与该系统非常接近的表达。注意， $A \in \mathbb{R}^{S \times H}$ 是输入令牌矩阵； $O \in \mathbb{R}^{S \times H}$ 是输出矩阵； $X \in \mathbb{R}^{E \times H \times D}$ 是专家权重的三维张量，其中， E 表示执行 GPU 的本地专家数， H 是嵌入维度， D 是 FFN 中间维度， S 是序列长度。 $T_\phi \in (\mathbb{N} \times \mathbb{R})^{E \times C}$ 是一个路由表数据结构，其中， $T_\phi(e, c) = (i, w)$ 表示槽位 c 处的标记 i 分发到专家 e 。 w 是组合权重 (公式 2)， C 是专家容量。 T_ϕ 的元组结构是一个实现细节。

$G_\phi \in \mathbb{R}^{S \times E}$ 捕获由门控产生的亲和度分数 (公式 3)。**Inter-actor interactions in FlashMoE.**

FlashMoE 以瓦片的粒度——张量的固定大小的分区——来分解 MoE 计算与通信，从而实现并行执行与任务的高效重叠。每个瓦片对应一个由 *taskdescriptor* 封装的离散工作单元。**Subscriber** 从其接收的远程瓦片数据包中解码这些任务描述符。同时，**Scheduler** 接收关于可用任务的通知，并将其分发给 **Processor** Actor 执行，这些 Actor 执行由这些任务定义的计算，即前馈网络 (FFN) 和专家合并操作。图 6 展示了 Actor 交互链，说明了 FlashMoE 如何强制执行 DMoE 功能依赖。

在 FlashMoE 中确定 tile 尺寸。 在 FlashMoE 中选择合适的 tile 尺寸对于确保高效的 GPU 利用率至关重要。过小的 tile 会使 GPU 得不到充分利用，而过大的 tile 会产生寄存器压力，导致寄存器溢出到本地内存，从而降低性能。经过仔细的参数扫描，我们选择了 tile 尺寸为 (128, 64)。我们的关键洞察是：增大 tile 宽度会显著提高每线程的寄存器使用量，可能触发代价高昂的溢出；在不调整线程数的情况下增大 tile 高度会提高每线程工作量，从而损害性能。将每块线程数提高到超过我们固定的 128 个线程后，会降低

并发块的数量，进而对SM占用率产生负面影响。更大的线程块尺寸还会增加块内同步（__线程块同步 屏障）的开销，进一步降低性能。因此，我们选择的tile尺寸在寄存器使用、共享内存约束和GPU占用率之间取得平衡，以实现最佳性能。

3.1 用于计算的任务抽象

计算算子。 FFN算子是一种在Transformer架构中广泛使用的标准逐位置前馈网络， [6], 由两个线性变换组成，中间通过一次非线性激活 ϕ （例如，GELU 或 ReLU）：

$$\text{FFN}(x) = W_2 \cdot \phi(xW_1 + b_1) + b_2 \quad (1)$$

这里， W_1 和 W_2 表示可学习权重矩阵， b_1 和 b_2 是偏置。专家合并操作，应用于诸如 GShard [29] 和 DeepSeek [1], 等架构，通过基于其亲和度分数计算加权组合来合并多个专家的输出：

$$C_i = \sum_{j=1}^k g_{i,e} \quad (2)$$

$$\mathbf{h}_i = \sum_{j=1}^k \frac{g_{i,e}}{C_i} \cdot \mathbf{h}_i^k \quad (3)$$

在这些方程中， $i \in 0, S-1$ 表示一个输入标记索引， $e = E_{i,k}$ 标识为标记 i 选择的第 k 个专家，而 $g_{i,e}$ 是亲和度分数，表示专家 e 对标记 i 的相关性。

统一任务抽象。 我们将 FFN 和合并操作统一到一个称为任务的通用抽象之下。任务在订阅者、调度器和处理器之间用于传递瓦片级工作，提供统一接口。形式上，任务描述符 $t \in \mathcal{T}$ 被定义为一个元组：

$$t = (\mathcal{M}, \star, \phi)$$

其中， $\mathcal{M}$ 是一组元数据（例如，设备ID，瓦片索引）， $\star$ 是一个二元张量操作（具体而言，矩阵乘法 $\cdot$ 或哈达玛积 $\odot$ ），并且 ϕ 是逐元素激活函数（例如，ReLU 或 恒等函数）。

我们定义一个任务 t ，其作用于输入张量 A 、 B 、 D ，并生成输出张量 C ，如下：

$$\mathcal{F}_t(A, B, C, D) := C \leftarrow \phi(A \star_t B + D) \quad (4)$$

The 算子 $\star_t$ （由 $\star$ 实例化）可能会根据任务元数据 $\mathcal{M}$ 而表现不同，一个 $A \star_t B$ 的结果会累加到 D 中。我们在 § E 中提供了任务元数据的一个示例。

在实践中，我们将由公式(4)定义的每个任务实现为一个单一融合的 `__device__` 修饰的函数，该函数由 **处理器** (算法2)在运行时调用。对于 t 的融合，意味着将 ϕ 以及随后的加法操作应用到存储二元算子 $\star_t$ 结果的寄存器上。为展示其灵活性，我们展示如何使用该任务框架来表示前馈网络和专家合并操作。注意，为简单起见，我们省略了矩阵乘法符号 $(\cdot)$ 。另外， ϕ_1 可以是任意激活函数，而 ϕ_2 是恒等函数。前馈网络表示为：

$$\begin{aligned} t_1 &= (\mathcal{M}, \cdot, \phi_1), \quad t_2 = (\mathcal{M}, \cdot, \phi_2), \\ \mathcal{F}_{t_1}(A, B_1, C_1, D_1) &:= C_1 \leftarrow \phi_1(AB_1 + D_1), \\ \mathcal{F}_{t_2}(C_1, B_2, C_2, D_2) &:= C_2 \leftarrow \phi_2(C_1B_2 + D_2). \end{aligned}$$

而专家合并操作形式化为：

$$\begin{aligned} t_3 &= (\mathcal{M}, \odot, \phi_2), \\ \mathcal{F}_{t_3}(A, S, C, C) &:= C \leftarrow \phi_2(A \odot S + C). \end{aligned}$$

3.2 用于跨GPU通信的对称张量布局

在单张GPU 设备内，FlashMoE 中的 Actor 通过 GPU 的内存子系统进行通信。具体而言，调度器和订阅者执行体通过快速共享内存交换数据，

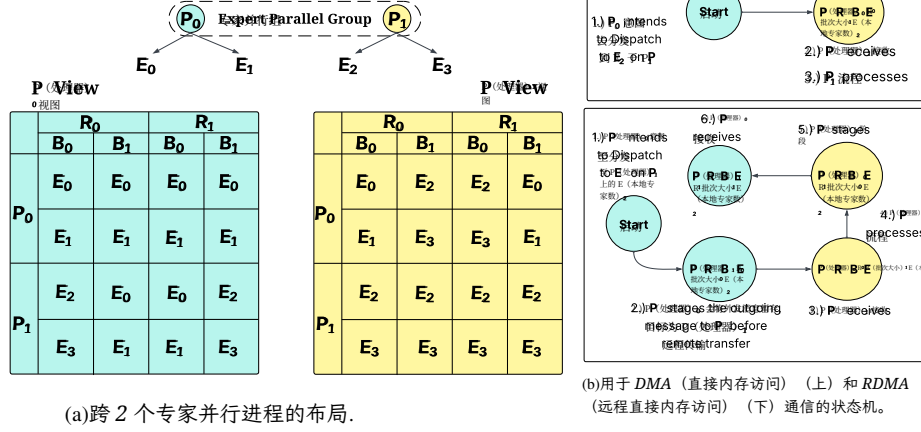


图 7: 对称张量布局

而其他 Actor 对通过全局内存进行通信。对于跨多个设备的通信, FlashMoE 使用 *device-initiated communication*, 利用单边 PGAS(分区全局地址空间) 编程模型 [30]。然而, 在不进行代价高昂的同步的情况下, 在 PGAS 中实现可扩展且正确的单边内存访问是一项众所周知的挑战 [1, 31]。我们通过一种可证明正确且可扩展的解决方案来应对这一挑战: 一种对称张量布局 L , 支持完全非阻塞内存访问。我们将 L 定义为:

$$L \in \mathbb{R}^{P \times R \times B \times E \times C \times H}$$

其中: P 是专家并行世界规模, R 表示通信轮次 (即, 两轮, 一轮用于令牌分发, 一轮用于合并), B 是暂存缓冲区数量, E 是本地专家数, C 是扩展的专家容量 (第 3.2.1 节), H 是令牌嵌入维度。我们实现非阻塞通信的核心洞见是时间缓冲。具体而言, 我们为底层令牌矩阵超额预留至少 $2 \cdot r$ 倍的内存, 其中 r 是依赖图中的通信轮次, 而因子 2 用于在每个通信轮次内分别为入站和出站数据设置独立缓冲区。对于 MoE 模型, 我们有 $2 \cdot r = 4$ 。这种少量的内存开销增加消除了单边数据传输期间对同步的需求。图 7b 说明了在该对称张量布局中如何对单元进行索引, 以及它们如何用于直接内存访问 (DMA) 和远程直接内存访问 (RDMA) 操作。正如定理 3.1 所强调的, 这种基于 L 的索引方案是允许完全非阻塞访问并省去同步的底层机制, 因为所有访问都是写操作无冲突。证明见第 C 节。

定理 3.1. 对称张量布局 L 是写写无冲突的。

为了构建 L , 我们从原始令牌缓冲区 $T \in \mathbb{R}^{S \times H}$ 开始, 其中, S 是序列长度, H 是令牌嵌入维度。我们首先将序列维度 S 重组为三个子维度, 分别表示专家容量 (C)、本地专家槽位 (E) 和专家并行世界规模 (W), 使得:

$$C \cdot E \cdot W = C \cdot E' = S', \quad \text{where } S' \geq S \text{ and } E' \geq E_W$$

在均匀专家分布的典型情况下 (如图 7a 所示), 我们有 $S' = S$ 和 $E' = E_W$, 其中 E_W 是模型中的专家总数。因此, 令牌缓冲区的大小为 $\text{Size}(T) = S' \cdot H$ 。在图 7a 中, 每个标记为 E_i (具有 $i \in \{0, \dots, 3\}$) 的单元都是一个大小为 (C, H) 的矩阵。在扩展先前工作 [29, 12], 的基础上, 我们引入了额外的时间维度 R (通信轮次) 和 B (暂存缓冲区)。每个通信轮次都有两个固定的暂存槽位: 一个用于出站令牌, 另一个用于入站令牌。每个槽位按维度 P 索引, 形成一个形状为 (S', H) 的张量。因此, 张量大小 $\text{Size}(L)$ 通常至少是原始令牌缓冲区大小的四倍; 在均匀专家分布的情况下, 恰好为四倍。实测发现 $\text{Size}(L) \approx 4 \cdot \text{Size}(T)$, 在热门模型的推理中造成约 $\leq 2\%$ 的内存容量的内存开销。我们在 § D 中给出了详细拆解。

3.2.1 就地填充以提升载荷效率

由于 MoE 调度中令牌分布具有动态性且不均匀 [32], GPU 通常接收的令牌少于其预定义的专家容量。现有 MoE 框架 [11] 通常在计算前用空令牌对这些缓冲区进行填充, 这会不必要地增加通信载荷并降低性能。相较之下, 我们提出就地填充, 直接在本地对称张量缓冲区内执行填充, 从而消除多余的网络通信。

作为参考, 正如图 7a 所示, 每个单元 E_i 的大小依据专家容量 C 确定。我们进一步将该容量对齐为可被瓦片块大小 $bM = 128$ 整除, 从而保证处理器线程在消费远程 token 时进行安全且对齐的内存读取。这种就地填充策略会略微增加 L 的内存占用, 如下所述:

$$Size(L) \approx \begin{cases} 4 \cdot Size(T), & \frac{S}{E} \geq bM \\ 4 \cdot \frac{bM \cdot E}{S} \cdot Size(T), & \text{otherwise} \end{cases}$$

4 评估

我们实现 (§ G) 并从五个指标评估 FlashMoE: **前向延迟**(§ 4.1)、**GPU 利用率**(§ 4.2)、**重叠效率**(§ 4.4)、**吞吐量**(§ 4.3)以及**专家可扩展性**(§ 4.5)。我们在一台配备 8 个 NVIDIA H100 80G GPU (通过 NVLink 互连)、125 GB 内存和 20 个 vCPU 的服务器上运行实验。我们使用 PyTorch 2.6.0、CUDA 12.8 和 Ubuntu 22.04。所有实验均使用 MoE Transformer 模型, 配置为 16 个注意力头、嵌入维度为 2048, 以及 FFN 中间层大小为 2048。所有实验均采用分布式数据并行 (DDP) 和专家并行。我们仅在单个 MoE 层上执行前向传播, 并在 32 次预热轮次后测量 32 次的平均运行时间。我们使用容量系数为 1.0 的 top-2 路由。我们将 FlashMoE 与多种当前最先进水平的 MoE 系统进行比较: (1)Comet [12], (2)FasterMoE [14], (3)Megatron-CUTLASS [13],以及 (4)Megatron-TE:

Megatron-LM 搭配 Transformer 引擎 [33]。Comet 依赖 cudaMemcpyPeerAsync [34], 而 FasterMoE 和 Megatron-LM 完全使用 NCCL 进行通信。

期望特性。 我们观察到 Comet 在 8 块 GPU 上表现出异常差的性能值, 因此我们在 8 块 GPU 的评测中排除其结果, 仅包含其在 ≤ 4 块 GPU 上的结果。我们使用 FP32 精度评测 FlashMoE, 而所有基线方法使用 FP16。这样做是因为 (1) FP16 调优不完整 (第 H 节), 以及 (2) 没有基线方法支持 FP32。注意, 这种精度差异使 FlashMoE 处于不利地位, 因为它使通信量和计算工作量加倍。

4.1 前向延迟

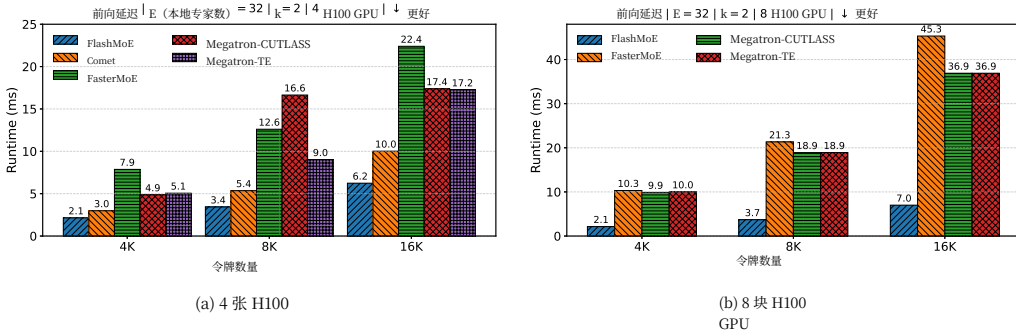


图8: 前向延迟随令牌数量在每个 GPU 上的增加而变化。

我们首先在 4 块 GPU 和 8 块 GPU 的设置上 (图8), 测量 FlashMoE 在不同序列长度下的前向延迟。FlashMoE 始终优于所有基线方法, 且在更长的序列长度下改进尤为显著。在 4 块 GPU 上, 它在 16K 个标记时相对于 Megatron-TE 可实现高达 4.6 倍的加速比, 相对于 FasterMoE 可达 2.6 倍。在 8 块 GPU 上, 优势更加明显; FlashMoE 仍保持低延迟, 相对于那些因标记缓冲区按比例增大而通信开销急剧上升、从而性能显著下降的基线方法, 获得高达 6.4 倍的加速比。

4.2 GPU 利用率

为量化 GPU 效率，我们在前向传播期间测量流式多处理器（SM）利用率（图9）。

FlashMoE 的平均 SM 利用率达到 93.17%，比 FasterMoE (9.67%) 高超过9倍，比 DeepEP+ Megatron-LM (13.55%) 高6.8倍，比 Megatron-TE (59.11%) 高4倍，以及比 Comet (42.31%) 高2.2倍。这种改进源于我们的全融合内核架构，以及对计算与通信任务的细粒度流水化。通过消除因内核启动导致的空闲间隙，并启用内核内任务调度，FlashMoE 确保 SM 在整个执行过程中始终忙于高效工作。

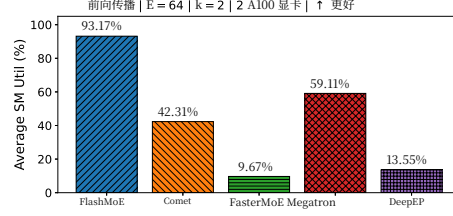


图9: SM 利用率，定义为：SM 至少有一个线程束在运行的周期数与总周期数之比 [17]。数值表示 100 次迭代的平均 SM 利用率。

4.3 吞吐量

如图10所示，FlashMoE 随 GPU 数量线性扩展，在 8 块 GPU 时达到 17.7 MTokens/s。其吞吐量比 Faster MoE 高出5.7倍，比 Megatron-TE 和 Megatron-CUTLASS 高出4.9倍。值得注意的是，尽管FlashMoE 完全使用 FP32 运行，而基线方法使用 FP16，仍取得了上述结果。这表明，Flash MoE 的设计并非通过利用低精度来消除吞吐量瓶颈，而是通过最大化硬件利用率并消除主机驱动的低效来实现的。

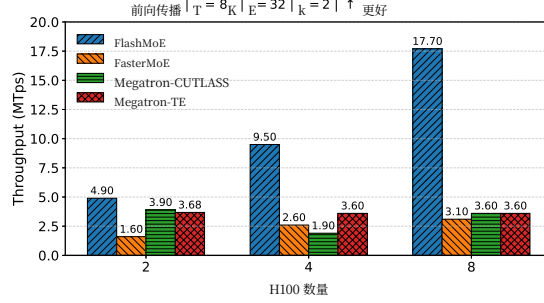
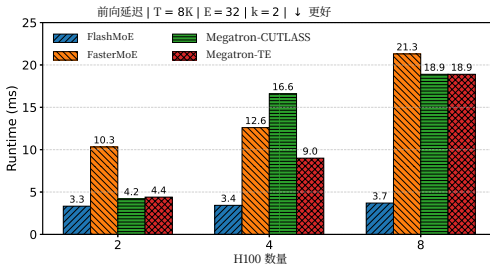
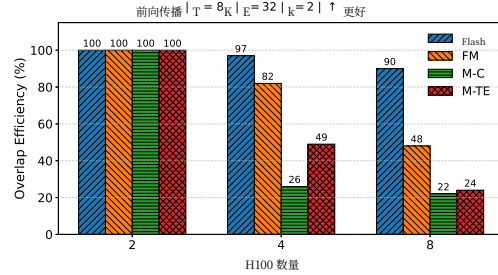


图10: 在扩展 GPU 数量时的吞吐量，计算方式为 $T \times N_{G_{\text{延迟}}}$ 。

4.4 重叠效率



(a) 随着 GPU 数量增加的延迟。



(b) 弱扩展效率

图11: 弱扩展效率。我们将重叠效率 O_e 定义为 $O_e = T(2)/T(N_G)$ ，其中 $T(N_G)$ 表示在 N_G GPU 下的延迟， $T(2)$ 表示在 2 块 GPU 下的延迟。

我们通过测量随着 GPU 数量增加（图11b）时的弱扩展效率，评估 FlashMoE 在多大程度上实现了通信与计算的重叠。我们注意到大多数基线方法无法在单张 GPU 上执行，因此使用 2 块 GPU 作为参考点。我们观察到 Megatron-CUTLASS 和 Megatron-TE 显著退化，在 ≥ 4 块 GPU 时重叠效率降至 50% 以下。FlashMoE 分别在 4 块 GPU 和 8 块 GPU 下将效率最高提升至3.88倍和4倍。图11a 进一步说明了这种效率，因为 FlashMoE 表现出稳定的前向延迟增长。这些结果证明，FlashMoE 的基于Actor的设计和异步数据移动实现了近乎理想的重叠。

4.5 专家可扩展性

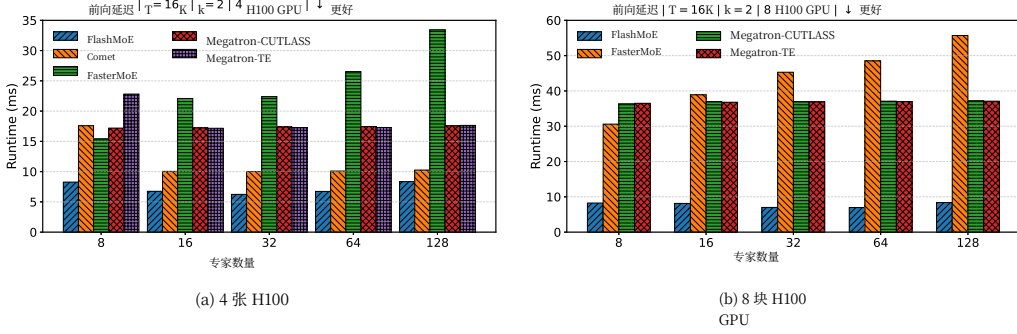


图12: 专家数量增加时的前向延迟。

我们分析在固定序列长度 ($T=16K$) 下, 随着专家数量增加 FlashMoE 的扩展性。请注意, 在所讨论的图中, x轴上的专家数量是跨所有GPU的总数。每个 GPU 获得该数值的 $1/8$ 。正如图12所示, 即使专家数量从 8 增长到 128, FlashMoE 仍能保持低且一致的延迟, 达到预期。相比之下, 基线方法由于内核启动开销增加而表现出超线性延迟增长。FlashMoE 在 4 张 H100 上可达4倍, 在 8 块 H100 GPU 上可达6.6倍, 二者均在 128 名专家条件下。FlashMoE 的负载高效的通信和由调度器驱动的内核内分发, 使其能够在不产生其他系统中所见通信与编排开销的情况下维持专家并行。这些结果进一步强化了 FlashMoE 在超稀疏的专家混合配置下的可扩展性。

5 局限性与未来工作

工程复杂性。 完全融合、持久化内核需要深厚的 GPU + 分布式系统专业知识; 未来工作可探索编译器/DSL 抽象以降低这一门槛。

FP16 低效。 由于调优不足, 我们的 FP16 路径 (第 H 节) 并不理想。我们预计借助自动调优的 GEMM (通用矩阵乘法) 算子 (如 cuBLASDx [35] 或 CUTLASS 构建器) 来弥补这一差距。

训练支持。 本工作面向推理; 要实现训练, 需要将反向计算和梯度通信与新的记录管理和任务描述符进行融合。

6 结论

我们提出了 FlashMoE, 这是首个将整个分布式混合专家算子融合到单一持久化 GPU 内核中的工作; 该内核通过 Actor 风格并发、warp 专用化和异步 (R)DMA 统一了计算、通信与调度。我们解决了以往系统中的两大瓶颈——由 CPU 管理的同步通信以及碎片化的多内核执行。实证结果显示, 对于分布式混合专家, FlashMoE 可实现最高 **6x** 加速比、**9x** 更高的 GPU 利用率, 以及**5.7x**吞吐量。展望未来, 我们看到了从 CPU 编排转向完全自治的 GPU 原生流水线的趋势——将这种融合方法扩展到训练及更广领域。

7 致谢

本研究由美国国家科学基金会 (NSF) 奖项 #2444537 和 ACE (JUMP 2.0 中七个中心之一) 资助; JUMP 2.0 是由美国国防高级研究计划局 (DARPA) 资助的半导体研究公司 (SRC) 项目。该工作还使用了国家能源研究科学计算中心的资源, 该中心是美国能源部科学办公室用户设施, 获得美国能源部科学办公室根据合同编号 DE-AC02-05CH11231 的支持, 并使用了 NERSC 奖项 ASCR-ERCAP0030076。我们在此致谢并感谢 Giulia Guidi 博士为我们提供这些 NERSC 超级计算资源的访问权限。

References

- [1] DeepSeek-AI. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- [2] Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, 2025. URL <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [3] Mosaic Research. Introducing dbrx: A new state-of-the-art open llm, 2024. URL <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>.
- [4] Snowflake AI Research. Snowflake arctic: The best llm for enterprise ai — efficiently intelligent, truly open, 2024. URL <https://www.snowflake.com/en/blog/arctic-open-efficient-foundation-language-models-snowflake/>.
- [5] OpenAI. GPT-OSS: Open-weight language models for reasoning and agentic tasks. <https://github.com/openai/gpt-oss>, August 2025. Models: gpt-oss-120b and gpt-oss-20b.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [7] Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatele. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th ACM International Conference on Supercomputing*, ICS ’23, page 203–214, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700569. doi: 10.1145/3577193.3593704. URL <https://doi.org/10.1145/3577193.3593704>.
- [8] Arjun Devraj, Eric Ding, Abhishek Vijaya Kumar, Robert Kleinberg, and Rachee Singh. Efficient allreduce with stragglers, 2025. URL <https://arxiv.org/abs/2505.23523>.
- [9] Juncal Liu, Jessie Hui Wang, and Yimin Jiang. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM ’23, page 486–498, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702365. doi: 10.1145/3603269.3604869. URL <https://doi.org/10.1145/3603269.3604869>.
- [10] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 74–86, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/339caf45a6fa281cae8adc6465343464-Paper-Conference.pdf.
- [11] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18332–18346. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/rajbhandari22a.html>.
- [12] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. In *MLSys ’25*. URL <https://arxiv.org/abs/2502.19811>.
- [13] NVIDIA. Megatron-lm, 2025. URL <https://github.com/NVIDIA/Megatron-LM?tab=readme-ov-file>. v0.11.0.
- [14] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 22)*, pages 120–134, 2022.

- [15] Michael Wendt and Joshua Wyatt. Getting started with CUDA graphs. <https://developer.nvidia.com/blog/cuda-graphs/>, 2019. Accessed: 2024-05-15.
- [16] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- [17] NVIDIA. NVIDIA Nsight Systems Metrics, . URL <https://docs.nvidia.com/nsight-systems/UserGuide/index.html?highlight=SM%2520active#available-metrics>.
- [18] NVIDIA. Nvidia nvshmem library (nvshmem), 2025. URL <https://docs.nvidia.com/nvshmem/api/index.html.v3.2.5>.
- [19] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, 2025. URL <https://github.com/NVIDIA/cutlass>.
- [20] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. Tutel: Adaptive mixture-of-experts at scale. In D. Song, M. Carbin, and T. Chen, editors, *Proceedings of Machine Learning and Systems*, volume 5, pages 269–287. Curran, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/file/5616d34cf8ff73942cfd5aa922842556-Paper-mlsys2023.pdf.
- [21] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '22*, page 120–134, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392044. doi: 10.1145/3503221.3508418. URL <https://doi.org/10.1145/3503221.3508418>.
- [22] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement. *Proc. ACM Manag. Data*, 1(1), May 2023. doi: 10.1145/3588964. URL <https://doi.org/10.1145/3588964>.
- [23] Shaohuai Shi, Xinglin Pan, Qiang Wang, Chengjian Liu, Xiaozhe Ren, Zhongzhe Hu, Yu Yang, Bo Li, and Xiaowen Chu. Schemoe: An extensible mixture-of-experts distributed training system with tasks scheduling. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys '24*, page 236–249, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704376. doi: 10.1145/3627703.3650083. URL <https://doi.org/10.1145/3627703.3650083>.
- [24] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Harnessing inter-gpu shared memory for seamless moe communication-computation fusion. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPOPP '25*, page 170–182, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714436. doi: 10.1145/3710848.3710868. URL <https://doi.org/10.1145/3710848.3710868>.
- [25] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.

- [26] Gul A. Agha. Actors: A model of concurrent computation in distributed systems. Technical report, 1985. MIT Artificial Intelligence Laboratory Technical Reports.
- [27] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. IJCAI’73, page 235–245, San Francisco, CA, USA, 1973. Morgan Kaufmann Publishers Inc.
- [28] Irene Greif. *SEMANTICS OF COMMUNICATING PARALLEL PROCESSES*. PhD thesis, Massachusetts Institute of Technology, 1975.
- [29] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.
- [30] Katherine Yelick, Dan Bonachea, Wei-Yu Chen, Phillip Colella, Kaushik Datta, Jason Duell, Susan L. Graham, Paul Hargrove, Paul Hilfinger, Parry Husbands, Costin Iancu, Amir Kamil, Rajesh Nishtala, Jimmy Su, Michael Welcome, and Tong Wen. Productivity and performance using partitioned global address space languages. In *Proceedings of the 2007 International Workshop on Parallel Symbolic Computation, PASCO ’07*, page 24–32, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595937414. doi: 10.1145/1278177.1278183. URL <https://doi.org/10.1145/1278177.1278183>.
- [31] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. Triton-distributed: Programming overlapping kernels on distributed ai systems with the triton compiler, 2025. URL <https://arxiv.org/abs/2504.19442>.
- [32] Quentin Anthony, Yuri Tokpanov, Paolo Glorioso, and Beren Millidge. Blackmamba: Mixture of experts for state-space models, 2024. URL <https://arxiv.org/abs/2402.01771>.
- [33] NVIDIA. Transformer engine, . URL <https://github.com/NVIDIA/TransformerEngine>.
- [34] Bytedance. Flux’s overlap performance is worse than non-overlap [4xrtx 4090], 2025. URL <https://github.com/bytedance/flux/issues/111#issuecomment-2822823236>.
- [35] NVIDIA. cublasdx, 2025. URL <https://docs.nvidia.com/cuda/cublasdx/>.
- [36] NERSC. Network - NERSC Documentation. <https://docs.nersc.gov/performance/network/>, 2025. [Accessed 23-05-2025].
- [37] C. Bell, D. Bonachea, R. Nishtala, and K. Yelick. Optimizing bandwidth limited problems using one-sided communication and overlap. In *Proceedings 20th IEEE International Parallel & Distributed Processing Symposium*, pages 10 pp.–, 2006. doi: 10.1109/IPDPS.2006.1639320.
- [38] Yuxin Chen, Benjamin Brock, Serban Porumbescu, Aydin Buluc, Katherine Yelick, and John Owens. Atos: A task-parallel gpu scheduler for graph analytics. In *Proceedings of the 51st International Conference on Parallel Processing, ICPP ’22*, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450397339. doi: 10.1145/3545008.3545056. URL <https://doi.org/10.1145/3545008.3545056>.
- [39] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 22)*, pages 402–416, 2022.
- [40] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 22)*, pages 93–106, 2022.

- [41] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 24)*, pages 178–191, 2024.
- [42] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D Sinclair. T3: Transparent tracking & triggering for fine-grained overlap of compute & collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 24)*, pages 1146–1164, 2024.
- [43] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024.
- [44] Weigao Sun, Zhen Qin, Weixuan Sun, Shidi Li, Dong Li, Xuyang Shen, Yu Qiao, and Yiran Zhong. CO2: Efficient distributed training with full communication-computation overlap. In *The Twelfth International Conference on Learning Representations (ICLR 24)*, 2024.
- [45] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. Better together: Jointly optimizing {ML} collective scheduling and execution planning using {SYNDICATE}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, 2023.
- [46] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. Harnessing inter-gpu shared memory for seamless moe communication-computation fusion. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 170–182, 2025.
- [47] Kishore Punniyamurthy, Khaled Hamidouche, and Bradford M Beckmann. Optimizing distributed ml communication with fused computation-collective operations. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–17, 2024.
- [48] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems (MLSys 23)*, 5:269–287, 2023.
- [49] Chenyu Jiang, Ye Tian, Zhen Jia, Shuai Zheng, Chuan Wu, and Yida Wang. Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping. In *Proceedings of Machine Learning and Systems (MLSys 24)*, pages 74–86, 2024.
- [50] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- [51] XAI. Grok-1. <https://huggingface.co/xai-org/grok-1>, 2024.
- [52] Snowflake. Snowflake arctic. <https://huggingface.co/Snowflake/snowflake-arctic-instruct>, 2024.
- [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [54] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- [55] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [Yes]

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [NA]

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All reported results in the evaluation section were obtained as the average of 32 executions preceded by 32 warmup runs.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: We do not foresee immediate social or ethical impacts, but we acknowledge that increased compute efficiency could amplify access to large-scale models, which raises general considerations around prevalent issues such as environmental cost of training, and responsible downstream use. We recommend that users of our system consider these factors when integrating it into broader ML applications.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [NA]

A Supplementary Motivation

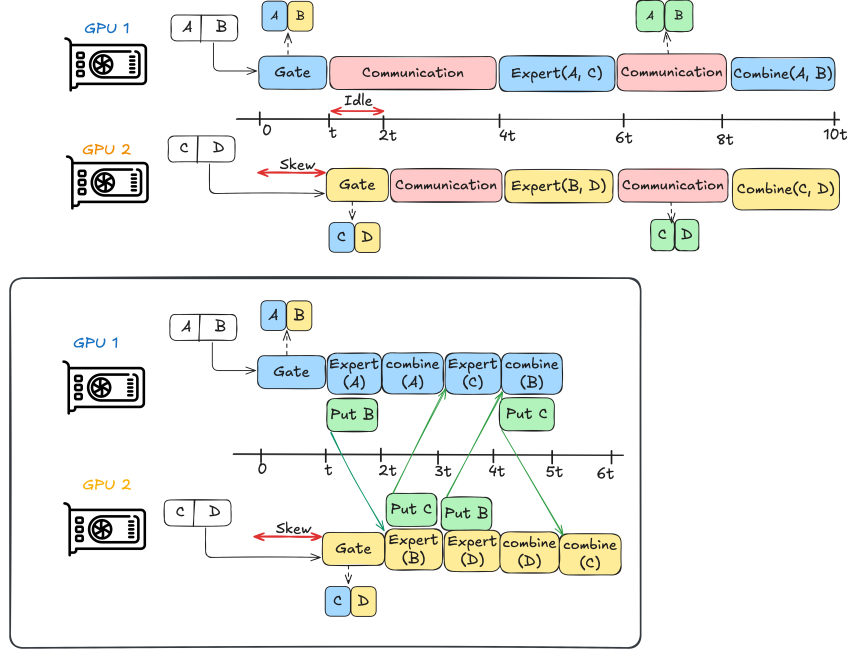


Figure 13: Overlapped Schedule (bottom) showing how idle time from the sequential schedule (top) is repurposed for computation. FlashMoE implements the overlapped schedule.

In Figure 14, we present empirical cumulative and raw distributions of *AlltoAll* kernel runtime from distributed training of a 1.3B GPT-3 MoE model across 32 A100 and 8 V100 GPUs. We use this result to motivate the severity and prevalence of straggler effects. In Figure 14b, we observe P95 communication performance degradation of **1.32X** when compared to the mean actual kernel time. This performance reduction is rather tame as the underlying hardware is a supercomputer well-tuned against “software jitter” [36]. However, we observe a more severe p95 performance loss of **11X** in a single-node Virtual Machine (VM). In line with prior HPC works [37, 38], we argue that obviating the inherent barrier in this synchronous collective communication would allow GPUs to repurpose this observed idle time for downstream computation as depicted in Figure 13.

Table 2: Straggler Delay within Synchronous *All-to-All* communication. We capture the distribution of delay induced by stragglers across many steps. Let **Actual Time** t_a denote the fastest kernel execution time across all GPUs, and **Total Time** t be the maximum recorded step time. We define *Delay* as the maximum difference between t and t_a . Note *Delay* is idle time. For the 1x8 V100, we profile 1750 steps and 600 steps for the 8x4 A100. See Figure 14 for the raw distribution.

System	# Nodes	# GPUs	Median	p95
Commercial VM (V100)	1	8	3.1x	11.4x
Supercomputer (A100)	8	32	1.09x	1.32x

B Related Work

Computation-Communication Overlap and Kernel Fusion. To reduce the communication overheads of synchronization in distributed DNN training, many research efforts have been focused on increasing the overlap of computation and communication. For generic Transformer-based models without MoE layers, many works [39–47] have provided insights and techniques to partition and schedule computation and communication operations, aimed at finer-grained overlapping. To address the challenges posed by *AlltoAll* communication and expert parallelism in MoE training, Tutel [48]

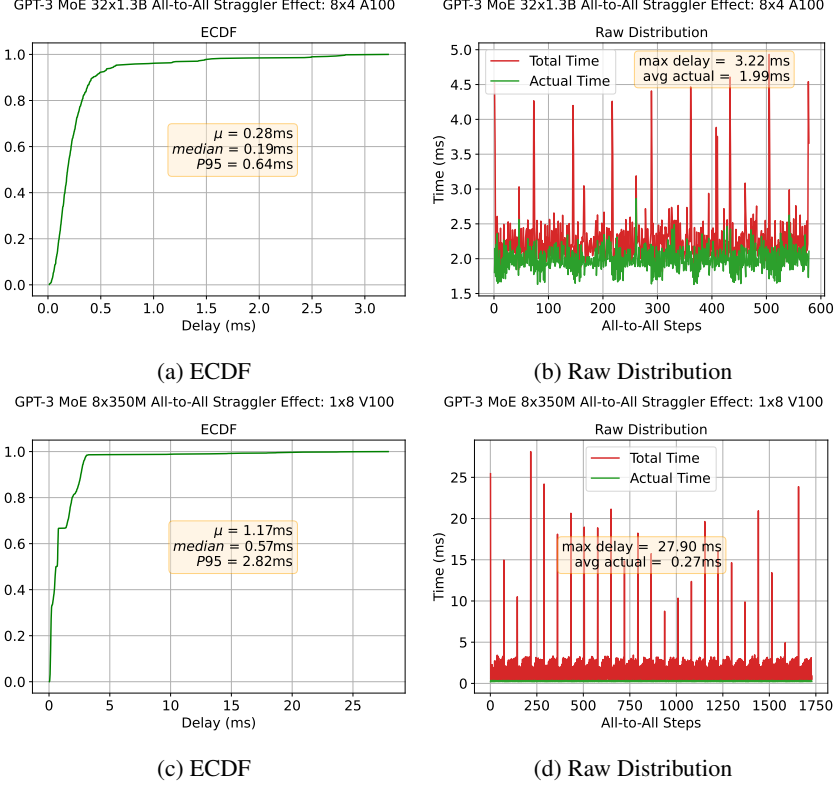


Figure 14: Straggler effect of synchronous *AlltoAll*. $M \times N$ A100 or V100 denotes N GPUs within a node across M nodes. Every GPU communicates with every other GPU per *AlltoAll* step. We capture the distribution of delay induced by stragglers across many steps. **Actual Time** t_a denotes the fastest kernel execution time across all GPUs, conversely **Total Time** t is the maximum recorded step time, while *Delay* is the maximum difference between t and t_a . Note *Delay* is idle time.

and FasterMoE [14] overlap *AlltoAll* with expert computation. Lancet [49] additionally enables both non-MoE computation in forward pass and weight gradient computation in backward pass to be overlapped with *AlltoAll*. Despite overlapping, the performance of these approaches is limited in practice due to blocking synchronous collective communication with barriers. In contrast, FlashMoE fundamentally eliminates these inefficiencies with asynchronous, device-initiated data transfers overlapped with tiled computation all *within a single kernel*. FlashMoE further differentiates itself from SOTA works like COMET [12] and DeepEP [1], which also use this form of kernel-initiated communication but at a coarse-grained granularity and without complete kernel fusion.

C Proof of Theorem 3.1

We begin with two necessary definitions vital to the proof.

Definition C.1. Define a write as $w(p_s, p_t, i)$, where p_s is the source process and i is an ordered tuple indicating the index coordinates for L residing on the target process p_t . A write-write conflict occurs when there exist at least two distinct, un-synchronized, concurrent writes $w_1(p_{s_1}, p_{t_1}, i_1)$ and $w_2(p_{s_2}, p_{t_2}, i_2)$, such that $p_{t_1} = p_{t_2}$ and index coordinates $i_1 = i_2$ but $p_{s_1} \neq p_{s_2}$.

Definition C.2. For any source process p_s , a valid index coordinate $i = (p^*, r, b, e, c)$ satisfies the following:

1. For inter-device writes, it must hold that $p^* = p_s$ and $b = 1$. Note this also applies to self-looping writes $w(p_t, p_t, i)$.

2. For any write $w(p_s, p_t, i)$, if $b = 0$, then $p_s = p_t$. This rule describes intra-device staging writes.

We restate Theorem 3.1 and outline its proof below.

Theorem C.1. *The symmetric tensor layout L is write-write conflict-free.*

Proof. As is the case for typical physical implementations, assume that each index coordinate i maps to a distinct memory segment in L . Next, we show by contradiction that no write-write conflicts can exist when accessing L using *valid* i . For simplicity, we only include the index coordinates when describing a write. Assume that there exist at least two writes $w_1(p_{s_1}, p_{t_1}, i_1)$, $w_2(p_{s_2}, p_{t_2}, i_2)$ with $p_{t_1} = p_{t_2}$ and valid destination coordinates i_1, i_2 , where $i_1 = i_2$ lexicographically and both are unpacked below.

$$i_1 = (p_1, r_1, b_1, e_1, c_1), i_2 = (p_2, r_2, b_2, e_2, c_2)$$

Note that intra-process writes always have distinct c_j coordinates, where $j \in \{0, C - 1\}$. For inter-process transfers, we have two cases.

Case 1: $p_{s_1} = p_{s_2}$

Here, w_1 and w_2 are identical operations. This contradicts the definition of a conflict, which requires that $p_{s_1} \neq p_{s_2}$. In practice, such repeat writes never even occur.

Case 2: $p_{s_1} \neq p_{s_2}$

To ensure validity for i_1 and i_2 , it is the case that $p_1 = p_{s_1}$ and $p_2 = p_{s_2}$. However, this implies that $i_1 \neq i_2$ yielding a contradiction as desired. $\square$

D Memory Overhead

We measure the GPU memory required for the symmetric tensor L and runtime bookkeeping state of FlashMoE. The memory overhead primarily depends on the tile size, expert capacity (EC), and the number of experts (E). Table 3 summarizes the memory overhead across recent MoE models [50–55] during inference, showing that FlashMoE maintains a modest and predictable memory footprint. In particular, the symmetric tensor (ST) accounts for at most 2.15% additional memory relative to the total inference memory requirements.

Table 3: Memory overhead of FlashMoE (tile size $bM = 128$, $Size(T) = \text{Tokens} \times 4\text{KB}$).

Model	Params	S	E	H	I	ST (GB)	Model (GB)	Overhead (%)
Moonlight-16B-A3B	16B	8K	64	2K	1.38K	0.25	59	0.49
Grok-1	314B	8K	8	6K	32K	0.75	1169	0.15
Snowflake-Arctic	479B	4K	128	7K	4.75K	1.75	1784	0.12
Qwen3-235B-A22B	235B	40K	128	4K	1.5K	3.00	875	0.38
Mixtral 8x7B	47B	32K	8	4K	14K	2.00	175	2.15
DeepSeek-V3	685B	160K	256	7K	2K	1.50	2551	0.11

E Task Implementation

```
1 #define GEMMs 2
2 struct __align__(16) Task {
3     const byte* aData;
4     array<const byte*, GEMMs> bData;
5     array<byte*, GEMMs> cData;
6     array<const byte*, GEMMs> dData;
7     byte* rcData;
8     uint64_t* flags;
9     uint M;
10    uint syncIdx;
11    uint tileIdx;
12    uint batchIdx;
13    uint peerIdx;
14    uint expertIdx;
15    uint isPeerRemote;
16    TaskType taskType;
17    uint16_t tileSize;
18    // Pad till 128-byte cache line
19    uint padding[6] = {};
20 }
```

Figure 15: *Task Struct*. $\text{TaskType} \in \{GEMM_0, GEMM_1, \text{Combine}\}$

F Actors

F.1 Processor

Algorithm 2: *Processor Actor*: executed by a block

```
1 begin
2    $tQ \leftarrow \text{GetTQ}()$ 
3    $signal \leftarrow 0$ 
4   // shared memory variables
5    $task \leftarrow \{\}$ 
6    $interrupt \leftarrow \text{False}$ 
7    $complete \leftarrow \text{False}$ 
8   while  $interrupt == \text{False}$  do
9     if  $warpId == 0$  then
10      if  $threadId == 0$  then
11         $\text{awaitTaskFromScheduler}(interrupt, signal)$ 
12         $\text{FencedNotifyRQ}(ready)$ 
13      end if
14       $\text{syncwarp}()$ 
15       $\text{warpReadTQ}(tQ, signal, task)$ 
16    end if
17     $\text{syncthread}()$ 
18    if  $interrupt == \text{False}$  then
19      switch  $task.Type$  do
20        case  $GEMM_0$  do
21          // fused GEMM, epilogue and async tile staging
22           $\text{fGET}(GEMM_0, task)$ 
23          if  $threadId == 0$  then
24             $complete \leftarrow \text{NotifyTileCompletion}()$ 
25          end if
26           $\text{syncthread}()$ 
27          if  $complete == \text{True}$  then
28             $\text{NotifySchedulerNextGEMM}(tQ)$ 
29          end if
30        end case
31        case  $GEMM_1$  do
32          // fused GEMM, epilogue and async tile transfer
33           $\text{fGET}(GEMM_1, task)$ 
34        end case
35        case  $Combine$  do
36           $\text{combine}(task)$ 
37        end case
38      end switch
39    end if
40  end while
41 end
```

F.2 Scheduler

Algorithm 3: *Scheduler Actor*: executed by one warp

```
1 begin
2    $scheduled \leftarrow 0$ 
3    $tTB \leftarrow 0$ 
4    $tqState \leftarrow \{\}$ 
5    $pTDB \leftarrow \text{GetProcessorDoorbell}()$ 
6    $sTDB \leftarrow \text{GetSubscriberDoorbell}()$ 
7    $taskBound \leftarrow \text{GetTaskBound}()$ 
8    $tTB \leftarrow \text{AtomicLoad}(taskBound)$ 
9   // circular buffer ready queue
10   $rQ \leftarrow \{\}$ 
11  // Populate ready queue with Processor ids
12  PopulateRQ( $rQ$ )
13  while  $scheduled < tTB$  do
14     $lt \leftarrow 0$ 
15    do in parallel
16      Sweep doorbells and populate observed task counts into  $tqState$ 
17      Aggregate locally observed task counts into  $lt$ 
18    end
19     $qS, taskTally \leftarrow 0$ 
20    // qS is the inclusive output
21    WarpInclusiveSum( $lt, qS, tasktally$ )
22    while  $tasktally > 0$  do
23      Repopulate  $rQ$  with ready processor ids
24      do in parallel
25        Starting at  $rQ[qS]$ , signal processors about task indices from  $tqState$ 
26      end
27    end while
28    if  $threadId == 0$  then
29       $tTB \leftarrow \text{AtomicLoad}(taskBound)$ 
30    end if
31     $tTB \leftarrow \text{WarpBroadcast}(tTB)$ 
32  end while
33  InterruptSubscribers()
34  InterruptProcessors()
35 end
```

F.3 Subscriber

Algorithm 4: *Subscriber Actor*: executed by three warps

Input: $T_\phi \in (\mathbb{R}^2)^{E \times C}$, $G_\phi \in \mathbb{R}^{S \times E}$, $O \in \mathbb{R}^{S \times H}$, $X \in \mathbb{R}^{E \times H \times D}$

```

1 begin
2    $interrupt \leftarrow \text{GetSharedInterrupt}()$ 
3    $flags \leftarrow \text{GetSymmetricFlags}()$ 
4    $tQ \leftarrow \text{GetTQ}()$ 
5   // Predefined upper bound on the number of tasks.
6   // We modulate this value to the actual task count computed
7   // dispatch signals received from peer GPUs
8    $taskBound \leftarrow \text{GetTaskBound}()$ 
9   while  $\text{AtomicLoad}(interrupt) == \text{False}$  do
10    // dispatch flags
11    do in parallel
12      Visit dispatch flags
13      Atomically retrieve signal
14      if Signal is set and flag is not visited then
15        Mark visited
16         $\text{SelfCorrectTaskBound}(taskBound, Signal)$ 
17        Enforce memory consistency before consuming packet
18        Decode packet into a set of  $GEMM_0$  task descriptors using  $X$ 
19        Write task descriptors to  $tQ$ 
20        Notify Scheduler of decoded tasks
21      end if
22    end
23    Advance flags by number of dispatch flags length
24    Atomically retrieve signal
25    // combine signals
26    do in parallel
27      Visit combine flags: one per tile
28      if Signal is set and flag is not visited then
29        Mark visited
30        Enforce memory consistency before consuming packet
31        Decode packet into a set of combine task descriptors using  $T_\phi, G_\phi, O$ 
32        Write task descriptors to  $tQ$ 
33        Notify Scheduler of decoded tasks
34      end if
35    end
36  end while
37 end

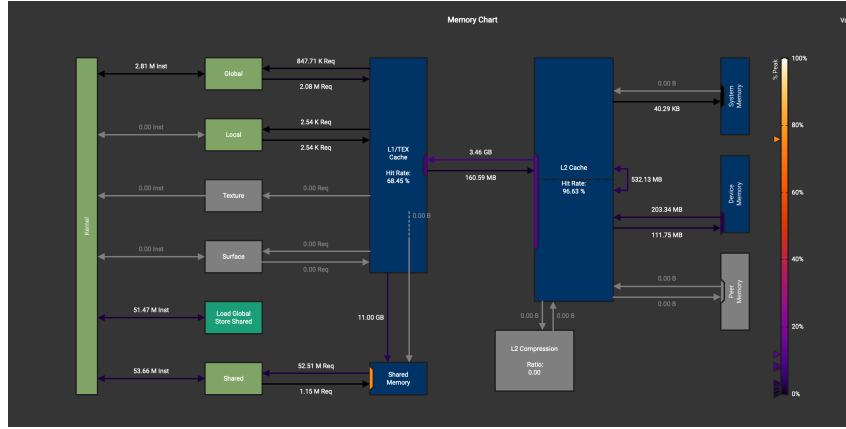
```

G Implementation

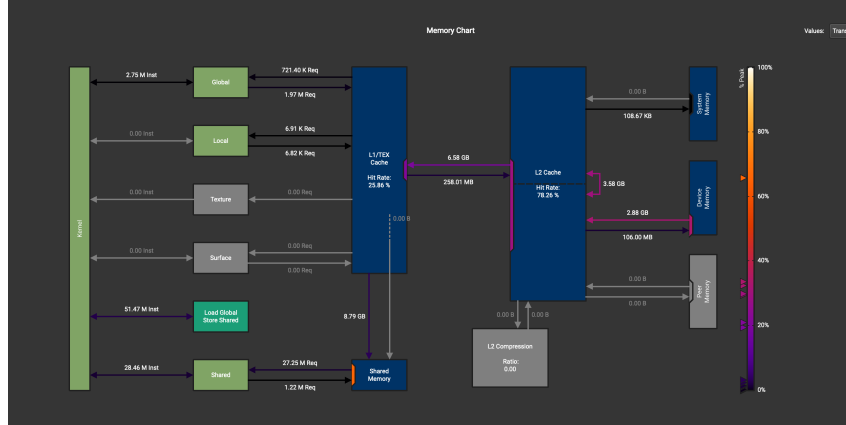
Table 4: Implementation metrics of FlashMoE.

Metric	Value
Total lines of code (CUDA/C++)	6820
Kernel stack frame size	0 B
Spill stores (per thread)	0
Spill loads (per thread)	0
Shared memory usage (per block)	46 KB
Registers per thread	255
Max active blocks per SM	2
Compilation time	53 seconds
Binary size	29 MB

H FP16 Memory Throughput



(a) Memory subsystem throughput for FP16



(b) Memory subsystem throughput for FP32

Figure 16: Here, we report the total A100 memory throughput for both FP16 (top) and FP32 (bottom) variants of FlashMoE. Notably, the FP16 implementation issues approximately $2\times$ more shared memory instructions compared to its FP32 counterpart under identical workloads. We attribute this inefficiency to suboptimal shared memory layouts in FlashMoE when operating on half-precision data. While this bottleneck is addressable through improved layout strategies, we leave its resolution to future work.